PurePy: A Functional Subset of Python

version 0.9.0

Nikolaus Huber[1], Dominic Orchard[2,1], Jacob Pake[2], Roly Perera[1,3]

[1] *Department of Computer Science and Technology, University of Cambridge, UK*
[2] *School of Computing, University of Kent, UK*
[3] *School of Computer Science, University of Bristol, UK*

PurePy is a pure functional subset of Python, providing a common functional language for use by researchers in programming languages and pedagogy, while providing a front-end familiar and accessible to the Python community. This specification defines a versioned standard with formal syntax, well-formedness rules based on a definite assignment analysis, and an operational semantics intended to preserve the meaning of well-formed PurePy programs in Python. By combining functional programming with Python-compatible surface syntax, PurePy aims to support portable implementations and lower-friction adoption of new language ideas for modelling, data processing, analysis, and visualisation.

1 Introduction

- Define subset of Python called PurePy, defined as a syntactic subset further restricted by a well-formedness predicate
- Define a semantics for PurePy such that the semantics of a PurePy program is identical to its semantics in Python
- Some well-formedness and evaluation rules could be given by a desugaring, but for now we avoid having two separate syntaxes

To do	
Reserved words	#9
Dataclasses	#8
Set and dict comprehensions	#52
Set and dict literals	#87
`import`	#53
Membership operator (`in`)	#80
Identity operator (`is`)	#81
Chained comparison	#82
`match` statement	#83
Enums	#86

Under consideration	
Destructuring assignment	#54
f-strings	#55
Default arguments	#56
`*args` and `**kwargs`	#57
Decorators	#58
Slicing	#59
Star patterns	#84
Or patterns	#85

2 Syntax

The syntax of the language is given in Figure 1. A *block* is a non-empty sequence of statements; either a top-level statement, a `def` body, or a branch of a conditional. A *mutual region* $d_1 \ldots d_{n+1}$ is a maximal contiguous non-empty sequence of `def` statements in a block: any non-`def` statement in that block, by

definition, separates mutual regions. Maximality is not enforced by the grammar in Figure 1; the parser is responsible for grouping consecutive `def`s into a single region. The well-formedness rules treat each region as one mutually-recursive binding (see §2.3.5).

2.1 Contexts

Contexts Γ, Δ are partial functions from sets of variables to a *definite assignment* status in $\mathbb{B} = \{\mathsf{tt}, \mathsf{ff}\}$, with tt indicating *definitely assigned* and ff indicating *not definitely assigned*. The definite assignment terminology is borrowed from Java [7, §16]. (Or think of tt and ff as degenerate types "any" and "none".) It is useful to represent contexts as total functions from the set of all variables to $\mathbb{B}_\bot = \mathbb{B} \cup \{\bot\}$, with $\bot$ indicating *undefined*. We write $\Gamma, x : a$ for the context that sends x to a and otherwise agrees with Γ, define $\mathrm{dom}(\Gamma) = \{x \mid \Gamma(x) \neq \bot\}$, and write $\varnothing$ for the context where all variables are undefined. For a set of variables X, write $\mathsf{tt}(X)$ for the context $\{x : \mathsf{tt} \mid x \in X\}$; similarly $\mathsf{ff}(X)$.

Context operations. Contexts admit a non-commutative monoid of sequential composition $\Gamma \cdot \Delta$ ("override") and an idempotent commutative semigroup of parallel composition $\oplus$ ("merge"). Together these form a *right near-semiring* [28]: $\cdot$ right-distributes over $\oplus$ (Lemma 1), but left distributivity does not hold. Both operations are defined pointwise via operations on $\mathbb{B}_\bot$:

Definition 1 (Override and merge)**.** *For $a, a' \in \mathbb{B}$:*

$$\bot \cdot a = a \cdot \bot = a \qquad\qquad \bot \oplus \bot = \bot$$
$$a \cdot a' = a' \qquad\qquad a \oplus \bot = \bot \oplus a = \mathsf{ff}$$
$$a \oplus a' = a \wedge a'$$

The $\cdot$ operation is right-biased and has $\bot$ as left and right unit. The $\oplus$ operation lacks a unit; when exactly one argument is defined, the result is ff; when both are defined, it returns the conjunction of their values in $\mathbb{B}$.

Lemma 1 (Right distributivity)**.**

$$(\Delta \oplus \Delta') \cdot \Gamma = (\Delta \cdot \Gamma) \oplus (\Delta' \cdot \Gamma).$$

Left distributivity ($\Gamma \cdot (\Delta \oplus \Delta') = (\Gamma \cdot \Delta) \oplus (\Gamma \cdot \Delta')$) does not hold. If it did, the following two programs would be equivalent, since sequential composition with a preceding assignment could be distributed into the branches of a conditional. But they are not: the left program is ill-formed, because a variable assigned before a conditional does not remain definitely assigned if only some branches assign it (see also §2.3.2).

```
def f(b):
    x = 1
    if b:
        x = 2
    else:
        pass
    return x  # error
```

```
def f(b):
    if b:
        x = 1
        x = 2
    else:
        x = 1
        pass
    return x  # ok
```

Context operations $\cdot$ and $\oplus$ lift to a right semiring over result types R, with $\mathsf{Assigns}\ \varnothing$ as left and right unit for $\cdot$ and $\mathsf{Returns}$ as unit for $\oplus$ and zero for $\cdot$.

Definition 2 (Override and merge on result types)**.**

$$\begin{aligned} \mathsf{Assigns}\ \Delta \oplus \mathsf{Assigns}\ \Delta' &= \mathsf{Assigns}\ (\Delta \oplus \Delta') \\ R \oplus \mathsf{Returns} = \mathsf{Returns} \oplus R &= R \end{aligned}$$

$$\begin{aligned} \mathsf{Assigns}\ \Delta \cdot \mathsf{Assigns}\ \Delta' &= \mathsf{Assigns}\ (\Delta \cdot \Delta') \\ R \cdot \mathsf{Returns} = \mathsf{Returns} \cdot R &= \mathsf{Returns} \end{aligned}$$

The $\mathsf{Returns} \cdot R$ case never arises because the well-formedness judgement prohibits unreachable statements (§2.3.7).

Lemma 2 (Right distributivity)**.**

$$(R \oplus R') \cdot S = (R \cdot S) \oplus (R' \cdot S)$$

Operators. The metavariable $\odot$ ranges over the supported binary operators and $\ominus$ over the supported unary operators, both enumerated in Figure 2. The grammar of Figure 1 is intentionally ambiguous on operator forms; disambiguation is by the precedence and associativity given in Figure 2, which follow Python.

2.2 Well-formedness

Auxiliary metafunctions. Three syntactic analyses support the well-formedness rules: $\mathrm{assigns}_{\mathsf{s}}(s)$, the set of variables assigned anywhere in s (without descending into nested function definitions, which have their own scope, so that $\mathrm{assigns}_{\mathsf{s}}(s) = \mathrm{dom}(\Delta)$ when $s : \mathsf{Assigns}\ \Delta$); $\mathrm{binds}(p)$, the set of variables introduced by a pattern p; and $\mathrm{captures}_{\mathsf{s}}(s)$, the set of variables from the enclosing scope that are captured by closures (lambdas or nested function definitions) within s, with companion $\mathrm{captures}_{\mathsf{e}}(e)$ on expressions. All are defined in Figure 3. The functions $\mathrm{assigns}_{\mathsf{s}}$ and $\mathrm{captures}_{\mathsf{s}}$ extend pointwise to blocks via $\mathrm{assigns}_{\mathsf{s}}(s \cdot b) = \mathrm{assigns}_{\mathsf{s}}(s) \cup \mathrm{assigns}_{\mathsf{s}}(b)$ (similarly $\mathrm{captures}_{\mathsf{s}}$), with the singleton block s identifying with the underlying statement.

Pattern subsumption. The preorder $p \sqsubseteq p'$, defined in Figure 4, holds when every value matched by p is also matched by p'. It is used by the well-formedness rule for `match` to rule out unreachable cases, and to check exhaustiveness: with the current pattern set, this reduces to requiring a catch-all (var or wildcard) case.

Well-formed pattern lists. A pattern list is well-formed (Figure 5) when each pattern is linear (no repeated variable names) and no earlier pattern subsumes a later one. The `match` well-formedness rules use this judgement to enforce reachability and linearity of case patterns.

Well-formedness judgement. Every well-formed statement either *definitely returns*, or *definitely assigns* a context Δ, which may be empty. The base cases are that `return e` definitely returns, and `x = e` definitely assigns $\{\texttt{x} : \mathsf{tt}\}$. Block well-formedness is given in Figure 6; expression well-formedness in Figure 7.

s	::=	`pass`	(Pass)
	|	$x = e$	(Assign)
	|	e	(ExprStmt)
	|	`assert` e	(Assert)
	|	`assert` e, e'	(AssertMsg)
	|	`return`	(ReturnNone)
	|	`return` e	(Return)
	|	`if` e `:` b `elif` e_1 `:` b_1 $\ldots$ `elif` e_n `:` b_n	(If)
	|	`if` e `:` b `elif` e_1 `:` b_1 $\ldots$ `elif` e_n `:` b_n `else :` b'	(IfElse)
	|	`match` e`:` `case` p_1`:` b_1 $\ldots$ `case` p_{n+1}`:` b_{n+1}	(Match)
	|	d_1 $\ldots$ d_{n+1}	(MutualRegion)
b	::=	s_1 $\ldots$ s_{n+1}	(Block)
d	::=	`def` $f(x_1, \ldots, x_n)$ `:` b	(Def)
e	::=	x	(Var)
	|	ℓ	(Literal)
	|	$e(e_1, \ldots, e_n)$	(Call)
	|	$\{x_1{:}e_1, \ldots, x_n{:}e_n\}$	(Dict)
	|	$e.x$	(Lookup)
	|	$e[e']$	(Subscript)
	|	$e \odot e'$	(BinOp)
	|	$\ominus e$	(UnaryOp)
	|	e_1 `if` e `else` e_2	(CondExpr)
	|	$[e_1, \ldots, e_n]$	(List)
	|	$(e_1, \ldots, e_n)$	(Tuple)
	|	$[e\ q_1\ \ldots\ q_n]$	(ListComp)
	|	`lambda` $x_1, \ldots, x_n$`:` e	(Lambda)
q	::=	`if` e	(Guard)
	|	`for` x `in` e	(Generator)
ℓ	::=	n	(Number)
	|	`True`	(True)
	|	`False`	(False)
	|	`None`	(None)
p	::=	ℓ	(PatLiteral)
	|	$-n$	(PatNegLit)
	|	x	(PatVar)
	|	_	(PatWildcard)
	|	$[p_1, \ldots, p_n]$	(PatList)
	|	$(p_1, \ldots, p_n)$	(PatTuple)
	|	p `as` x	(PatAs)
Γ, Δ	::=	$\{x_1{:}\mathbb{B}, \ldots, x_n{:}\mathbb{B}\}$	
R, S	::=	Returns	
	|	Assigns Δ	

Figure 1: Syntax of statements, expressions and contexts

Level	Operators	Arity	Associativity
1 (lowest)	`or`	binary	left
2	`and`	binary	left
3	`not`	unary	—
4	`==` `!=` `<` `<=` `>` `>=`	binary	left
5	`+` `-`	binary	left
6	`*` `/` `//` `%`	binary	left
7	`+` `-`	unary	—
8 (highest)	`**`	binary	right

Figure 2: Operator precedence (lowest binds loosest); follows Python.

2.3 Examples

2.3.1 Assignment as definition

Although PurePy is pure and all variable usage is well-scoped, local variables are introduced using an *assignment statement*, for compatibility with Python. If a variable is definitely assigned in any branch of a conditional, then for that variable to be usable after the conditional, every other branch of the conditional must either definitely return, or definitely assign that variable too. Thus the following are both well-formed:

```
if a < b:
   comp = "smaller"
else:
   comp = "bigger"
return comp
```

```
if a < b:
   return "smaller"
else:
   comp = "bigger"
return comp
```

The definite assignment set for a conditional that has at least one branch which definitely assigns is calculated by merging the definite assignment sets for all branches which definitely assign, using the $\oplus$ operator on environments.

2.3.2 Local assignment in conditional clauses

For flexibility, we permit *branch-local* variables. The assignment of `x` below (left) is permitted, and can be used locally in that branch of the conditional:

$$
\begin{aligned}
\mathrm{assigns_s}(\texttt{pass}) &= \varnothing \\
\mathrm{assigns_s}(x = e) &= \{x\} \\
\mathrm{assigns_s}(e) &= \varnothing \\
\mathrm{assigns_s}(\texttt{assert}\ e, e') &= \varnothing \\
\mathrm{assigns_s}(\texttt{return}) &= \varnothing \\
\mathrm{assigns_s}(\texttt{return}\ e) &= \varnothing \\
\mathrm{assigns_s}(\texttt{if}\ e_1 :\ b_1\ \texttt{elif}\ e_2 :\ b_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ b_{n+1}) &= \mathrm{assigns_s}(\vec{b}) \\
\mathrm{assigns_s}(\texttt{if}\ e_1 :\ b_1\ \texttt{elif}\ e_2 :\ b_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ b_{n+1}\ \texttt{else} :\ b_{n+2}) &= \mathrm{assigns_s}(\vec{b}) \\
\mathrm{assigns_s}(\texttt{def}\ f_1(\vec{x}_1) :\ b_1 \cdot \ldots \cdot \texttt{def}\ f_{n+1}(\vec{x}_{n+1}) :\ b_{n+1}) &= \{f_1, \ldots, f_{n+1}\} \\
\mathrm{assigns_s}(\texttt{match}\ e\texttt{: case}\ p_1\texttt{:}\ b_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ b_{n+1}) &= \textstyle\bigcup_i \left(\mathrm{binds}(p_i) \cup \mathrm{assigns_s}(b_i)\right)
\end{aligned}
$$

$$
\begin{aligned}
\mathrm{binds}(\ell) &= \varnothing \\
\mathrm{binds}(-n) &= \varnothing \\
\mathrm{binds}(x) &= \{x\} \\
\mathrm{binds}(_) &= \varnothing \\
\mathrm{binds}([p_1, \ldots, p_n]) &= \textstyle\bigcup_i \mathrm{binds}(p_i) \\
\mathrm{binds}((p_1, \ldots, p_n)) &= \textstyle\bigcup_i \mathrm{binds}(p_i) \\
\mathrm{binds}(p\ \texttt{as}\ x) &= \mathrm{binds}(p) \cup \{x\}
\end{aligned}
$$

$$
\begin{aligned}
\mathrm{captures_s}(\texttt{pass}) &= \varnothing \\
\mathrm{captures_s}(x = e) &= \mathrm{captures_e}(e) \\
\mathrm{captures_s}(e) &= \mathrm{captures_e}(e) \\
\mathrm{captures_s}(\texttt{assert}\ e, e') &= \mathrm{captures_e}(e) \cup \mathrm{captures_e}(e') \\
\mathrm{captures_s}(\texttt{return}) &= \varnothing \\
\mathrm{captures_s}(\texttt{return}\ e) &= \mathrm{captures_e}(e) \\
\mathrm{captures_s}(\texttt{if}\ e_1 :\ b_1\ \texttt{elif}\ e_2 :\ b_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ b_{n+1}) &= \mathrm{captures_e}(\vec{e}) \cup \mathrm{captures_s}(\vec{b}) \\
\mathrm{captures_s}(\texttt{if}\ e_1 :\ b_1\ \texttt{elif}\ e_2 :\ b_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ b_{n+1}\ \texttt{else} :\ b_{n+2}) &= \mathrm{captures_e}(\vec{e}) \cup \mathrm{captures_s}(\vec{b}) \\
\mathrm{captures_s}(\texttt{def}\ f_1(\vec{x}_1) :\ b_1 \cdot \ldots \cdot \texttt{def}\ f_{n+1}(\vec{x}_{n+1}) :\ b_{n+1}) &= \textstyle\bigcup_i \left(\mathrm{fv_s}(b_i) \setminus (\{\vec{x}_i\} \cup \mathrm{assigns_s}(b_i))\right) \\
&\quad \setminus \{f_1, \ldots, f_{n+1}\} \\
\mathrm{captures_s}(\texttt{match}\ e\texttt{: case}\ p_1\texttt{:}\ b_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ b_{n+1}) &= \mathrm{captures_e}(e) \cup \\
&\quad \textstyle\bigcup_i \left(\mathrm{captures_s}(b_i) \setminus \mathrm{binds}(p_i)\right)
\end{aligned}
$$

$$
\begin{aligned}
\mathrm{captures_e}(x) &= \varnothing \\
\mathrm{captures_e}(\texttt{lambda}\ \vec{x}\texttt{:}\ e) &= \mathrm{fv_e}(e) \setminus \{\vec{x}\}
\end{aligned}
$$

Figure 3: Assigned, bound, and captured variables (remaining expression cases of $\mathrm{captures_e}$ are the obvious pointwise union).

$p \sqsubseteq p'$

sub-lit
$$\frac{}{\ell \sqsubseteq \ell}$$

sub-neg-lit
$$\frac{}{-n \sqsubseteq -n}$$

sub-var
$$\frac{}{p \sqsubseteq x}$$

sub-wild
$$\frac{}{p \sqsubseteq _}$$

sub-list-list
$$\frac{p_i \sqsubseteq q_i \quad (\forall i)}{[p_1, \ldots, p_n] \sqsubseteq [q_1, \ldots, q_n]}$$

sub-list-tuple
$$\frac{p_i \sqsubseteq q_i \quad (\forall i)}{[p_1, \ldots, p_n] \sqsubseteq (q_1, \ldots, q_n)}$$

sub-tuple-list
$$\frac{p_i \sqsubseteq q_i \quad (\forall i)}{(p_1, \ldots, p_n) \sqsubseteq [q_1, \ldots, q_n]}$$

sub-tuple-tuple
$$\frac{p_i \sqsubseteq q_i \quad (\forall i)}{(p_1, \ldots, p_n) \sqsubseteq (q_1, \ldots, q_n)}$$

sub-as-l
$$\frac{p \sqsubseteq q}{p \texttt{ as } x \sqsubseteq q}$$

sub-as-r
$$\frac{p \sqsubseteq q}{p \sqsubseteq q \texttt{ as } y}$$

Figure 4: Pattern p is subsumed by pattern p'.

$\vdash p_1 \cdot \ldots \cdot p_{n+1}$

pat-list
$$\frac{\text{variables in } p_i \text{ distinct} \quad (\forall i) \qquad p_i \not\sqsubseteq p_j \quad (\forall i)(\forall j < i)}{\vdash p_1 \cdot \ldots \cdot p_n}$$

Figure 5: Pattern list $p_1 \cdot \ldots \cdot p_n$ is well-formed

$\Gamma \vdash b : R$

pass

$$\frac{}{\Gamma \vdash \texttt{pass} : \mathsf{Assigns}\ \varnothing}$$

assign

$$\frac{\Gamma \vdash e \qquad x \notin \mathrm{captures_e}(e)}{\Gamma \vdash x = e : \mathsf{Assigns}\ \{x : \mathsf{tt}\}}$$

expr-stmt

$$\frac{\Gamma \vdash e}{\Gamma \vdash e : \mathsf{Assigns}\ \varnothing}$$

assert

$$\frac{\Gamma \vdash e}{\Gamma \vdash \texttt{assert}\ e : \mathsf{Assigns}\ \varnothing}$$

assert-msg

$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash \texttt{assert}\ e, e' : \mathsf{Assigns}\ \varnothing}$$

return-none

$$\frac{}{\Gamma \vdash \texttt{return} : \mathsf{Returns}}$$

return

$$\frac{\Gamma \vdash e}{\Gamma \vdash \texttt{return}\ e : \mathsf{Returns}}$$

if

$$\frac{\Gamma \vdash e_i \quad (\forall i) \qquad \Gamma \vdash b_i : R_i \quad (\forall i)}{\Gamma \vdash \texttt{if}\ e_1 :\ b_1\ \texttt{elif}\ e_2 :\ b_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ b_{n+1} : R_1 \oplus \ldots R_{n+1} \oplus \ldots \oplus \mathsf{Assigns}\ \varnothing}$$

if-else

$$\frac{\Gamma \vdash e_i \quad (\forall i) \qquad \Gamma \vdash b_i : R_i \quad (\forall i)}{\Gamma \vdash \texttt{if}\ e_1 :\ b_1\ \texttt{elif}\ e_2 :\ b_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ b_{n+1}\ \texttt{else} :\ b_{n+2} : R_1 \oplus \ldots \oplus R_{n+2}}$$

mutual

$$\frac{\Gamma \cdot \mathsf{tt}(\vec{f}) \cdot \mathsf{tt}(\vec{x}_i) \cdot \mathsf{ff}(\mathrm{assigns_s}(b_i) \setminus \vec{x}_i) \vdash b_i : R_i \quad (\forall i) \qquad f_1, \ldots, f_{n+1}\ \text{distinct}}{\Gamma \vdash \texttt{def}\ f_1(\vec{x}_1) :\ b_1 \cdot \ldots \cdot \texttt{def}\ f_{n+1}(\vec{x}_{n+1}) :\ b_{n+1} : \mathsf{Assigns}\ \mathsf{tt}(\vec{f})}$$

match

$$\frac{\begin{array}{c}\Gamma \vdash e \qquad \Gamma \cdot \mathsf{tt}(\mathrm{binds}(p_i)) \vdash b_i : R_i \quad (\forall i) \\ S_i = \mathsf{Assigns}\ \mathsf{tt}(\mathrm{binds}(p_i)) \cdot R_i \quad (\forall i) \qquad \vdash p_1 \cdot \ldots \cdot p_{n+1} \qquad p_{n+1} \in \{x, _\}\end{array}}{\Gamma \vdash \texttt{match}\ e\texttt{: case}\ p_1\texttt{:}\ b_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ b_{n+1} : S_1 \oplus \ldots \oplus S_{n+1}}$$

match-partial

$$\frac{\begin{array}{c}\Gamma \vdash e \qquad \Gamma \cdot \mathsf{tt}(\mathrm{binds}(p_i)) \vdash b_i : R_i \quad (\forall i) \\ S_i = \mathsf{Assigns}\ \mathsf{tt}(\mathrm{binds}(p_i)) \cdot R_i \quad (\forall i) \qquad \vdash p_1 \cdot \ldots \cdot p_{n+1} \qquad p_{n+1} \notin \{x, _\}\end{array}}{\Gamma \vdash \texttt{match}\ e\texttt{: case}\ p_1\texttt{:}\ b_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ b_{n+1} : S_1 \oplus \ldots \oplus S_{n+1} \oplus \mathsf{Assigns}\ \varnothing}$$

seq

$$\frac{\Gamma \vdash s : \mathsf{Assigns}\ \Delta \qquad \Gamma \cdot \Delta \vdash b : R \qquad \mathrm{captures_s}(s) \cap \mathrm{assigns_s}(b) = \varnothing}{\Gamma \vdash s \cdot b : \mathsf{Assigns}\ \Delta \cdot R}$$

Figure 6: Block b is well-formed in Γ

$\Gamma \vdash e$

$$\frac{\Gamma(x) = \mathsf{tt}}{\Gamma \vdash x}\ \textsc{var} \qquad \frac{\Gamma, x_1 : \mathsf{tt}, \ldots, x_n : \mathsf{tt} \vdash e}{\Gamma \vdash \texttt{lambda}\ x_1, \ldots, x_n \texttt{:}\ e}\ \textsc{lambda}$$

Figure 7: Expression e is well-formed in Γ

```
def foo():
  z = 6
  if b:
    x = 3
    y = x + 1
  else:
    y = 0
  return z
```

(a) Branch-local definition of `x`

```
def foo():
  x = 6
  if b:
    x = 3
    y = x + 1
  else:
    y = 0
  return x # error: partially defined
```

(b) Now with use of `x` after the conditional

However such variables only appear to be branch-local. The definition of $\oplus$ actually ensures that a variable assigned in any branch is considered assigned by the conditional; however unless it is assigned in *every* branch that does not definitely return, it will be bound to ff in the corresponding environment. Any attempt to refer to such a variable is illegal, as it refers to a variable which may not be assigned. (Thus the right-hand program is ill-formed, despite the enclosing `x = 6`, because in the `return x` statement, the inner assignment shadows the the outer one, and the inner assignment is only partial.) If we were to allow the program on the right, there would be no straightforward way to assign it a semantics which is both pure, and in agreement with Python, since in Python the function returns `3` in the case where `b` is `True`.

2.3.3 Function-level scoping of local variables

In Python, all local variables are scoped to the entire function body, not just from the point of assignment onwards. The $\mathrm{assigns_s}$ function in the mutual rule reflects this: all variables assigned anywhere in the function body are introduced at ff before the body is checked. This means the following program is statically rejected by PurePy, whereas Python rejects it at runtime with an `UnboundLocalError`:

```
y = 7

def f():
    x = y  # error: y is local (due to assignment below)
    y = 8

f()
```

Because `y` appears on the left-hand side of an assignment in `f`, it is in $\mathrm{assigns_s}(b)$ for the body of `f`, and is therefore introduced at ff at the top of the function. The reference to `y` in `x = y` then fails the well-formedness check, since $\Gamma(\texttt{y}) = \mathsf{ff}$ rather than $\Gamma(\texttt{y}) = \mathsf{tt}$.

2.3.4 Reassignment of captured variables

Once a variable has been captured by a closure, it cannot be reassigned in the enclosing scope, because its Python semantics would depend on mutation. Consider the following program:

```
def f():
    x = 5
    def g():
        return x
    x = 6
    return x + g()
```

In Python, `g` closes over the *mutable binding* of `x`, so `g()` returns `6` and the result is `12`. In a pure semantics where `x = 6` shadows the earlier binding, `g()` would return `5` and the result would be `11`. To preserve agreement with a subset of Python, we exclude this program from PurePy: the side condition in the seq rule requires that no variable captured by s is reassigned in the rest of the block b. Here `x` is captured by the definition of `g` and reassigned by `x = 6`, so the program is rejected. This is analogous to Java's requirement that local variables captured by a lambda must be *effectively final* [7, §15.27.2]: local variables are allocated on the stack, and so only the value of a local variable can be captured by a closure.

The same issue arises when a variable is captured and reassigned in the *same* statement:

```
x = 5
def x():
    return x
x()  # Python: returns the function (late binding); pure: returns 5
```

The side condition on the assign rule rejects programs like this, requiring that the variable being bound is not captured by the expression. The mutual rule, by contrast, binds each f_i in the typing context for every body b_i, so the function names may legitimately appear in $\mathrm{captures}_{\mathsf{s}}$ of those bodies. This is what permits self-recursion (the singleton-region case) and mutual recursion within a region.

2.3.5 Mutual recursion

The mutual rule binds the names $f_1, \ldots, f_{n+1}$ of a mutual region $d_1 \ldots d_{n+1}$ simultaneously. Each body b_i is checked in a context that includes all the f_j as definitely assigned, so any f_j may reference any other f_k within the region (the singleton case $n = 0$ permits self-recursion). The names in a region must be distinct.

Inserting any non-`def` statement between two `def`s splits them into separate regions, breaking the mutual reference:

```
def even(n):
    if n == 0:
        return True
    return odd(n - 1)

x = even(0)  # intervening statement separates the regions

def odd(n):
    if n == 0:
        return False
```

```
    return even(n - 1)
```

This program is rejected statically: `even` forms a singleton region in which `odd` is not yet bound, so the reference to `odd` in its body fails the var rule. Python accepts it: the call `even(0)` does not actually reach `odd`, and Python's late binding means `odd` is only looked up when needed.

Each branch of a conditional may contain its own mutual regions, subject to the usual definite assignment rules. The following is well-formed because both branches define both names:

```
b = True

if b:
    def f():
        return g()
    def g():
        return "via mutual region"
else:
    def f():
        return "f independent"
    def g():
        return "g independent"

print(f())
print(g())
```

A region containing two definitions of the same name is rejected directly by the mutual rule's distinct-names side condition:

```
def g():
    return 0

def f():
    return g()
def g():     # error: g defined twice in the same region
    return 1
```

(Python accepts this; late binding makes `f`'s call to `g` resolve to the second `def g`.)

Wrapping the second `def f; def g` pair in a conditional sidesteps the duplicate-name issue: the outer `def g` and the inner `def g` are now in different blocks, so they no longer fall in the same region. One might still expect a captured-variable problem, since the inner `def g` could be seen as rebinding the outer `g` that `f` captures. No such problem arises:

```
def g():
    return 0

if b:
    def f():
        return g()
    def g():
        return 1
```

The inner `def f; def g` form a branch-local mutual region, so `f`'s body resolves `g` to its sibling rather than to the outer `g`. The two are bound simultaneously, so `f` cannot be called before the inner `g` exists, and the value `f` sees is the same as the value Python's late binding finds at call time.

2.3.6 Pattern variables escape the match

A variable bound by a `case` pattern is in scope after the `match` statement, just as a variable assigned inside a conditional branch is in scope after the conditional. Python treats matching this way; PurePy follows.

```
v = 5
match v:
    case x:
        pass
print(x)    # prints 5
```

Here the catch-all `case x:` binds `x` in the body's scope, and the binding survives into the enclosing scope. Definite assignment after the match follows the same merge logic as a conditional: if every case binds the variable (or returns), it is definitely assigned afterwards; otherwise it is not (see also §2.3.2).

2.3.7 Unreachable code

A non-empty sequence of statements returns only if the last statement returns, and all preceding statements assign. Thus the following is illegal:

```
def foo(x):
  return x + 1
  return x    # error: unreachable
```

2.3.8 Conditionals without `else` clauses

An `if` statement without an `else` clause is equivalent to an `if` statement with an `else` clause whose body is a `pass` statement; such a statement never returns.

2.3.9 Implicit `None`

A `return` statement without an expression is equivalent to `return None`.

2.3.10 Implicit `return`

A function whose body s assigns rather than returns is equivalent to a function whose body is s followed by `return None`. This ensures every function returns, even if it is not explicitly specified.

2.3.11 `assert` without a message

An `assert e` statement without a message is equivalent to `assert e, ""`.

$$
\begin{array}{lcll}
v & ::= & \ell & \text{(Literal)} \\
 & | & [v_1, \ldots, v_n] & \text{(List)} \\
 & | & (v_1, \ldots, v_n) & \text{(Tuple)} \\
 & | & \langle \rho, \vec{x}, e \rangle & \text{(LambdaClosure)} \\
 & | & \langle \rho, \vec{d}, i \rangle & \text{(DefClosure)} \\
\\
\rho & ::= & \{x_1 \mapsto v_1, \ldots, x_n \mapsto v_n\} & \text{(Environment)} \\
\\
r & ::= & \mathsf{assigns}\ \rho & \\
 & | & \mathsf{returns}\ v &
\end{array}
$$

Figure 10: Values, environments and evaluation results

2.4 Unsupported Python features

The following Python features are excluded from PurePy as they are incompatible with purity or functional programming:

- `while` loops, `for` loops, `break`, `continue`
- `async`/`await`
- `global`, `nonlocal`
- `del`
- `try`/`except`/`raise`
- Augmented assignment (`+=`, `-=`, etc.)
- Walrus operator (`:=`)
- `yield`/`yield from`
- `class` definitions (dataclasses may be added; see §2 to-do list)
- `with` statements

3 Operational semantics

For a statement s, the result $\mathsf{assigns}\ \rho$ represents *normal completion* of s, with accumulated environment ρ; the result $\mathsf{returns}\ v$ represents *abrupt completion* of s with return value v. Composing statements sequentially composes their results according to the following partial monoid, analogous to sequential composition of result types:

$$
\begin{aligned}
\mathsf{assigns}\ \rho \cdot \mathsf{assigns}\ \rho' &= \mathsf{assigns}\ (\rho \cdot \rho') \\
\mathsf{assigns}\ \rho \cdot \mathsf{returns}\ v &= \mathsf{returns}\ v
\end{aligned}
$$

with $\mathsf{assigns}\ \varnothing$ as left and right unit. Function definitions delimit the scope of these effects, so as black boxes functions are always pure.

The pattern-matching judgement $p, v \rightsquigarrow \rho$ is a deterministic partial function on (p, v). We write $p, v \not\rightsquigarrow$ (with no right-hand side) for $\nexists \rho.\, p, v \rightsquigarrow \rho$, i.e. matching is undefined.

4 Related work

Subsets of programming languages have been defined for various purposes in the research literature:

$$\begin{aligned}
\text{dispatch}(v, \varepsilon) &= (\varnothing, \texttt{pass}) \\
\text{dispatch}(v, (\texttt{case } p\texttt{: } b) \cdot \vec{c}) &= (\rho, b) && \text{if } p, v \rightsquigarrow \rho \\
\text{dispatch}(v, (\texttt{case } p\texttt{: } b) \cdot \vec{c}) &= \text{dispatch}(v, \vec{c}) && \text{if } p, v \not\rightsquigarrow
\end{aligned}$$

$\rho, b \Rightarrow r$

eval-pass

$$\frac{}{\rho, \texttt{pass} \Rightarrow \textsf{assigns } \varnothing}$$

eval-assign

$$\frac{\rho, e \Rightarrow v}{\rho, x = e \Rightarrow \textsf{assigns } \{x \mapsto v\}}$$

eval-expr-stmt

$$\frac{\rho, e \Rightarrow v}{\rho, e \Rightarrow \textsf{assigns } \varnothing}$$

eval-assert

$$\frac{\rho, e \Rightarrow v \qquad v \neq \texttt{False}}{\rho, \texttt{assert } e \Rightarrow \textsf{assigns } \varnothing}$$

eval-assert-msg

$$\frac{\rho, e \Rightarrow v \qquad v \neq \texttt{False}}{\rho, \texttt{assert } e, e' \Rightarrow \textsf{assigns } \varnothing}$$

eval-return

$$\frac{\rho, e \Rightarrow v}{\rho, \texttt{return } e \Rightarrow \textsf{returns } v}$$

eval-mutual

$$\frac{d_i = \texttt{def } f_i(\vec{x}_i)\texttt{: } b_i \quad (\forall i) \qquad v_i = \langle \rho, \vec{d}, i \rangle \quad (\forall i)}{\rho, \vec{d} \Rightarrow \textsf{assigns } \{\vec{f} \mapsto \vec{v}\}}$$

eval-if

$$\frac{\rho, e_j \Rightarrow \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \texttt{True} \qquad \rho, b_i \Rightarrow r}{\rho, \texttt{if } e_1\texttt{: } b_1 \texttt{ elif } e_2\texttt{: } b_2 \texttt{ } \ldots \texttt{ elif } e_{n+1}\texttt{: } b_{n+1} \texttt{ else: } b_{n+2} \Rightarrow r}$$

eval-else

$$\frac{\rho, e_j \Rightarrow \texttt{False} \quad (\forall j \in \{1, \ldots, n+1\}) \qquad \rho, b_{n+2} \Rightarrow r}{\rho, \texttt{if } e_1\texttt{: } b_1 \texttt{ elif } e_2\texttt{: } b_2 \texttt{ } \ldots \texttt{ elif } e_{n+1}\texttt{: } b_{n+1} \texttt{ else: } b_{n+2} \Rightarrow r}$$

eval-match

$$\frac{\rho, e \Rightarrow v \qquad \text{dispatch}(v, \texttt{case } p_1\texttt{: } b_1 \cdot \ldots \cdot \texttt{case } p_{n+1}\texttt{: } b_{n+1}) = (\rho', b) \qquad \rho \cdot \rho', b \Rightarrow r}{\rho, \texttt{match } e\texttt{: case } p_1\texttt{: } b_1 \texttt{ } \ldots \texttt{ case } p_{n+1}\texttt{: } b_{n+1} \Rightarrow r}$$

eval-seq

$$\frac{\rho, s \Rightarrow \textsf{assigns } \rho' \qquad \rho \cdot \rho', b \Rightarrow r}{\rho, s \cdot b \Rightarrow \textsf{assigns } \rho' \cdot r}$$

Figure 11: Block evaluation

$$\mathrm{unwrap}(\textsf{returns } v) = v$$
$$\mathrm{unwrap}(\textsf{assigns } \rho) = \texttt{None}$$

$\rho, e \Rightarrow v$

eval-var

$$\frac{}{\rho, x \Rightarrow \rho(x)}$$

eval-lambda

$$\frac{}{\rho, \texttt{lambda}\ x_1, \ldots, x_n\texttt{:}\ e \Rightarrow \langle \rho, \vec{x}, e \rangle}$$

eval-call-lambda

$$\frac{\rho, e \Rightarrow \langle \rho', \vec{x}, e' \rangle \qquad \rho, \vec{e} \Rightarrow \vec{u} \qquad \rho' \cdot \{\vec{x} \mapsto \vec{u}\}, e' \Rightarrow v}{\rho, e(e_1, \ldots, e_n) \Rightarrow v}$$

eval-call-def

$$\frac{\begin{array}{c}\rho, e \Rightarrow \langle \rho', \vec{d}, j \rangle \qquad d_i = \texttt{def}\ f_i(\vec{x}_i)\texttt{:}\ b_i \quad (\forall i) \\ v_i = \langle \rho', \vec{d}, i \rangle \quad (\forall i) \qquad \rho, \vec{e} \Rightarrow \vec{u} \qquad \rho' \cdot \{\vec{f} \mapsto \vec{v}\} \cdot \{\vec{x}_j \mapsto \vec{u}\}, b_j \Rightarrow r\end{array}}{\rho, e(e_1, \ldots, e_n) \Rightarrow \mathrm{unwrap}(r)}$$

Figure 12: Expression evaluation

$p, v \rightsquigarrow \rho$

pat-lit

$$\frac{}{\ell, \ell \rightsquigarrow \varnothing}$$

pat-neg-lit

$$\frac{}{-n, -n \rightsquigarrow \varnothing}$$

pat-var

$$\frac{}{x, v \rightsquigarrow \{x \mapsto v\}}$$

pat-wild

$$\frac{}{_, v \rightsquigarrow \varnothing}$$

pat-list-list

$$\frac{p_i, v_i \rightsquigarrow \rho_i \quad (\forall i)}{[p_1, \ldots, p_n], [v_1, \ldots, v_n] \rightsquigarrow \rho_1 \uplus \ldots \uplus \rho_n}$$

pat-list-tuple

$$\frac{p_i, v_i \rightsquigarrow \rho_i \quad (\forall i)}{[p_1, \ldots, p_n], (v_1, \ldots, v_n) \rightsquigarrow \rho_1 \uplus \ldots \uplus \rho_n}$$

pat-tuple-list

$$\frac{p_i, v_i \rightsquigarrow \rho_i \quad (\forall i)}{(p_1, \ldots, p_n), [v_1, \ldots, v_n] \rightsquigarrow \rho_1 \uplus \ldots \uplus \rho_n}$$

pat-tuple-tuple

$$\frac{p_i, v_i \rightsquigarrow \rho_i \quad (\forall i)}{(p_1, \ldots, p_n), (v_1, \ldots, v_n) \rightsquigarrow \rho_1 \uplus \ldots \uplus \rho_n}$$

pat-as

$$\frac{p, v \rightsquigarrow \rho}{p\ \texttt{as}\ x, v \rightsquigarrow \rho \uplus \{x \mapsto v\}}$$

Figure 13: Pattern matching

Even though the syntax of programming languages if often described precisely, essentially none of the languages in mainstream use today formally define their behaviour. Therefore, various well-defined subsets of programming languages were constructed to equip languages with formal (often even mechanised) semantics. Examples of this include work on Java [10], OCaml [25], JavaScript [5, 8, 20, 21], PHP [6], Rust [12, 13], and even lower-level languages and intermediate representations such as WebAssembly [29, 30, 31] and LLVM-IR [32, 33]. A particular success story of this line of work is the formalisation of a substantial subset of the C programming language, called Clight [4], which lead to the development of a verified compiler [17]. In a similar manner, the CakeML project [15] produced a verified implementation of a fragment of the ML language [18], based on previous efforts [16] of mechanising its semantics. Previous work on formalising the semantics of Python include an embedding [9] within the $\mathbb{K}$ semantics framework [24], a definition of structural operational semantics [14], and a reduction of most of the language to a core calculus [22].

A different line of work concerns the delineation of language subsets for a particular purpose. A prominent example is the *Source* family of languages [3], which are progressively extended subsets of JavaScript specifically geared towards introducing first-year computer science students to the foundations of computing. It is based on a new edition of a classic introductory text book [1], in which all program examples (which had previously been in Scheme) are illustrated in JavaScript. By successively increasing the allowed language fragment with each chapter (corresponding to different versions of the Source language), it allows to put focus on the particular aspects the authors are trying to convey, while disallowing students with prior knowledge of programming to take short-cuts by utilising more complex language features. The authors of [3] also remark, that the choice of JavaScript as a language was a pragmatic one: A previous study [26] showed that *industrial relevance* is perceived as the second-most important factor in choosing a programming language for introductory courses by educators in Australasia and the UK.

Likewise, various subsets of Python have been defined for different purposes in the past. ChocoPy [19] describes a subset specifically defined as a source language for the final project in courses on compiler construction, as most students are already familiar with the Python language by the time they take such a course. By restricting the language to a statically typed fragment, most student compilers can produce code that usually even outperforms the CPython reference interpreter. In a similar manner, Restricted Python (RPython) [2] was defined as a strictly typed subset of Python, which removes some of the more dynamic features of the language. As part of the PyPy project [23], this allows RPython to be more easily compiled to common language runtimes, such as JVM and CLI/.NET.

We are aware of only one other work [27] specifically targetting a functional subset of Python, with the express intent of using it as a teaching tool. While we share a similar goal, our language fragment encloses more of the overall Python language, as their work doesn't allow for control structures such as conditionals or loops.

While it is not our goal to change Python towards a more functional programming language, it is interesting to notice that there are movements within the Python community for inclusion of programming principles commonly found in functional languages. An example of this is the inclusion of *lambda expressions* already in Python 1.0, and a more recent PEP draft [11] advocating for *deeply immutable objects*.

References

[1] H. Abelson, G.J. Sussman, M. Henz, T. Wrigstad, and J. Sussman. *Structure and Interpretation of Computer Programs: JavaScript Edition*. MIT Electrical Engineering and Computer Science. MIT Press, 2022.

[2] Davide Ancona, Massimo Ancona, Antonio Cuni, and Nicholas D. Matsakis. RPython: A step towards reconciling dynamically and statically typed OO languages. In *Proceedings of the 2007 Symposium on Dynamic Languages*, pages 53–64, Montreal Quebec Canada, October 2007. ACM.

[3] Boyd Anderson, Martin Henz, Kok-Lim Low, and Daryl Tan. Shrinking JavaScript for CS1. In *Proceedings of the 2021 ACM SIGPLAN International Symposium on SPLASH-E*, pages 87–96, Chicago IL USA, October 2021. ACM.

[4] Sandrine Blazy and Xavier Leroy. Mechanized Semantics for the Clight Subset of the C Language. *Journal of Automated Reasoning*, 43(3):263–288, October 2009.

[5] Martin Bodin, Arthur Charguéraud, Daniele Filaretti, Philippa Gardner, Sergio Maffeis, Daiva Naudziuniene, Alan Schmitt, and Gareth Smith. A trusted mechanised JavaScript specification. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 87–100, San Diego California USA, January 2014. ACM.

[6] Daniele Filaretti and Sergio Maffeis. An Executable Formal Semantics of PHP. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Alfred Kobsa, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Demetri Terzopoulos, Doug Tygar, Gerhard Weikum, and Richard Jones, editors, *ECOOP 2014 – Object-Oriented Programming*, volume 8586, pages 567–592. Springer Berlin Heidelberg, Berlin, Heidelberg, 2014.

[7] James Gosling, Bill Joy, Guy L. Steele Jr., Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman. *The Java Language Specification, Java SE 24 Edition*. Oracle, 2025.

[8] Arjun Guha, Claudiu Saftoiu, and Shriram Krishnamurthi. The essence of javascript. In *Proceedings of the 24th European Conference on Object-Oriented Programming*, ECOOP'10, pages 126–150, Maribor, Slovenia, 2010. Springer-Verlag.

[9] Dwight Guth. *A Formal Semantics of Python 3.3*. PhD thesis, University of Illinois at Urbana-Champaign, 2013.

[10] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight Java: A minimal core calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems*, 23(3):396–450, May 2001.

[11] Matthew Johnson, Matthew Parkinson, Sylvan Clebsch, Fridtjof Stoldt, and Tobias Wrigstad. PEP 795 – Deep Immutability for Efficient Sharing and Concurrency Safety, June 2025.

[12] Ralf Jung, Hoang-Hai Dang, Jeehoon Kang, and Derek Dreyer. Stacked borrows: An aliasing model for Rust. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–32, January 2020.

[13] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):1–34, January 2018.

[14] Maximilian A. Köhl. An Executable Structural Operational Formal Semantics for Python, September 2021.

[15] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. CakeML: A verified implementation of ML. *ACM SIGPLAN Notices*, 49(1):179–191, January 2014.

[16] Daniel K. Lee, Karl Crary, and Robert Harper. Towards a mechanized metatheory of standard ML. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 173–184, Nice France, January 2007. ACM.

[17] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, July 2009.

[18] Robin Milner, Robert Harper, David MacQueen, and Mads Tofte. *The Definition of Standard ML*. The MIT Press, 2 edition, May 1997.

[19] Rohan Padhye, Koushik Sen, and Paul N. Hilfinger. ChocoPy: A programming language for compilers courses. In *Proceedings of the 2019 ACM SIGPLAN Symposium on SPLASH-E*, pages 41–45, Athens Greece, October 2019. ACM.

[20] Daejun Park, Andrei Stefănescu, and Grigore Roşu. KJS: A complete formal semantics of JavaScript. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 346–356, Portland OR USA, June 2015. ACM.

[21] Joe Gibbs Politz, Matthew J. Carroll, Benjamin S. Lerner, Justin Pombrio, and Shriram Krishnamurthi. A tested semantics for getters, setters, and eval in JavaScript. In *Proceedings of the 8th Symposium on Dynamic Languages*, pages 1–16, Tucson Arizona USA, October 2012. ACM.

[22] Joe Gibbs Politz, Alejandro Martinez, Mae Milano, Sumner Warren, Daniel Patterson, Junsong Li, Anand Chitipothu, and Shriram Krishnamurthi. Python: The full monty. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*, pages 217–232, Indianapolis Indiana USA, October 2013. ACM.

[23] Armin Rigo and Samuele Pedroni. PyPy's approach to virtual machine construction. In *Companion to the 21st ACM SIGPLAN Symposium on Object-oriented Programming Systems, Languages, and Applications*, pages 944–953, Portland Oregon USA, October 2006. ACM.

[24] Grigore Roșu and Traian Florin Șerbănuță. An overview of the K semantic framework. *The Journal of Logic and Algebraic Programming*, 79(6):397–434, August 2010.

[25] Remy Seassau, Irene Yoon, Jean-Marie Madiot, and François Pottier. Formal Semantics and Program Logics for a Fragment of OCaml. *Proceedings of the ACM on Programming Languages*, 9(ICFP):128–159, August 2025.

[26] Simon, Raina Mason, Tom Crick, James H. Davenport, and Ellen Murphy. Language Choice in Introductory Programming Courses at Australasian and UK Universities. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, pages 852–857, Baltimore Maryland USA, February 2018. ACM.

[27] Rajshekhar Sunderraman. Functional Python Programming in Introductory Computer Science Courses, December 2025.

[28] W. G. van Hoorn and B. van Rootselaar. Fundamental notions in the theory of seminearrings. *Compositio Mathematica*, 18(1-2):65–78, 1967.

[29] Conrad Watt. *Mechanising and Evolving the Formal Semantics of WebAssembly: The Web’s New Low-Level Language*. PhD thesis, Apollo - University of Cambridge Repository, 2021.

[30] Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, and Philippa Gardner. Two Mechanisations of WebAssembly 1.0. In Marieke Huisman, Corina Păsăreanu, and Naijun Zhan, editors, *Formal Methods*, volume 13047, pages 61–79. Springer International Publishing, Cham, 2021.

[31] Conrad Watt, Maja Trela, Peter Lammich, and Florian Märkl. WasmRef-Isabelle: A Verified Monadic Interpreter and Industrial Fuzzing Oracle for WebAssembly. *Proceedings of the ACM on Programming Languages*, 7(PLDI):100–123, June 2023.

[32] Yannick Zakowski, Calvin Beck, Irene Yoon, Ilia Zaichuk, Vadim Zaliva, and Steve Zdancewic. Modular, compositional, and executable formal semantics for LLVM IR. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–30, August 2021.

[33] Jianzhou Zhao, Santosh Nagarakatte, Milo M.K. Martin, and Steve Zdancewic. Formalizing the LLVM intermediate representation for verified program transformations. *ACM SIGPLAN Notices*, 47(1):427–440, January 2012.